MLDL
PRACTICAL 10

Name: Payal Dara
Class: D15C
Roll no: 56
Batch: C

AIM:-

Explore and implement Autoencoders for image denoising

---

1. Dataset Source

Dataset Name: MNIST Dataset
Source: TensorFlow / Keras Built-in Dataset

---

2. Dataset Description

The MNIST dataset contains grayscale images of handwritten digits (0–9).

Dataset Characteristics

- Training Samples: 60,000
- Testing Samples: 10,000
- Image Size: 28 × 28
- Channels: 1
- Type: Image data

Objective

- Remove noise from images using autoencoders

---

3. Mathematical Formulation of Autoencoder

Autoencoder consists of encoder and decoder.

Encoding

$z=f(Wx+b)$

Decoding

$\hat{x}=g(W'z+b')$

Loss Function (MSE)

$Loss=\frac{1}{n}\sum(x-\hat{x})^2$

---

4. Algorithm Limitations (Autoencoder)

- Requires large dataset
- May not generalize well
- Computationally expensive
- Sensitive to architecture

---

5. Methodology / Workflow

Dataset → Add Noise → Autoencoder Model
→ Training → Reconstruction → Visualization

---

6. Performance Analysis

Metrics

- Loss (MSE)
- Visual quality of output images

Interpretation

- Lower loss → better reconstruction
- Clearer output images → better denoising

7. Hyperparameter Tuning (Autoencoder)

Parameters Tuned

- Number of neurons
- Number of layers
- Epochs
- Batch size

Effect

- Improves reconstruction quality
- Reduces noise more effectively
- Enhances model performance

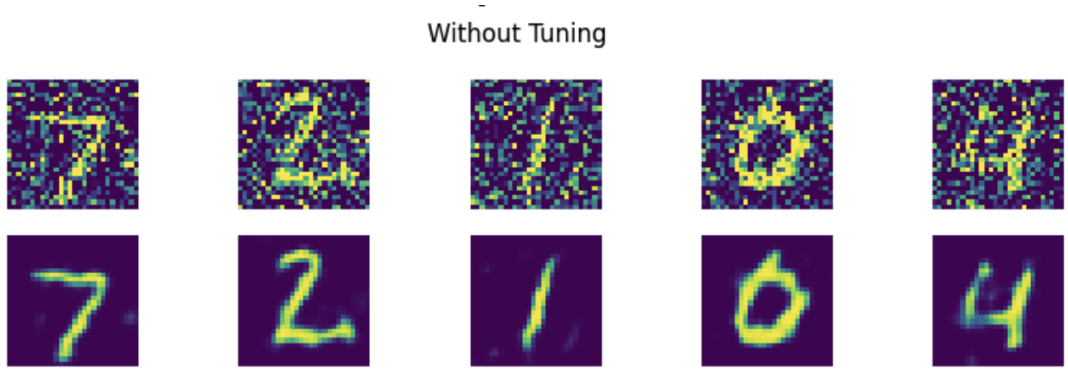
Code:-

Without tuning:-

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras import layers, models

# Load dataset
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()

# Normalize
x_train = x_train / 255.0
x_test = x_test / 255.0

# Add noise
noise_factor = 0.5
x_train_noisy = x_train + noise_factor * np.random.normal(size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(size=x_test.shape)

# Clip values
```

```
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)

# Reshape
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)
x_train_noisy = x_train_noisy.reshape(-1, 28, 28, 1)
x_test_noisy = x_test_noisy.reshape(-1, 28, 28, 1)

# Build Autoencoder
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', padding='same',
input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2), padding='same'),
    layers.Conv2D(32, (3,3), activation='relu', padding='same'),
    layers.UpSampling2D((2,2)),
    layers.Conv2D(1, (3,3), activation='sigmoid', padding='same')
])

model.compile(optimizer='adam', loss='mse')

# Train
history1 = model.fit(x_train_noisy, x_train, epochs=5, batch_size=128,
                     validation_data=(x_test_noisy, x_test))

# Predict
decoded_imgs1 = model.predict(x_test_noisy)

# Show images
plt.figure(figsize=(10,4))
for i in range(5):
    # Noisy
    plt.subplot(3,5,i+1)
    plt.imshow(x_test_noisy[i].reshape(28,28))
    plt.axis('off')
```

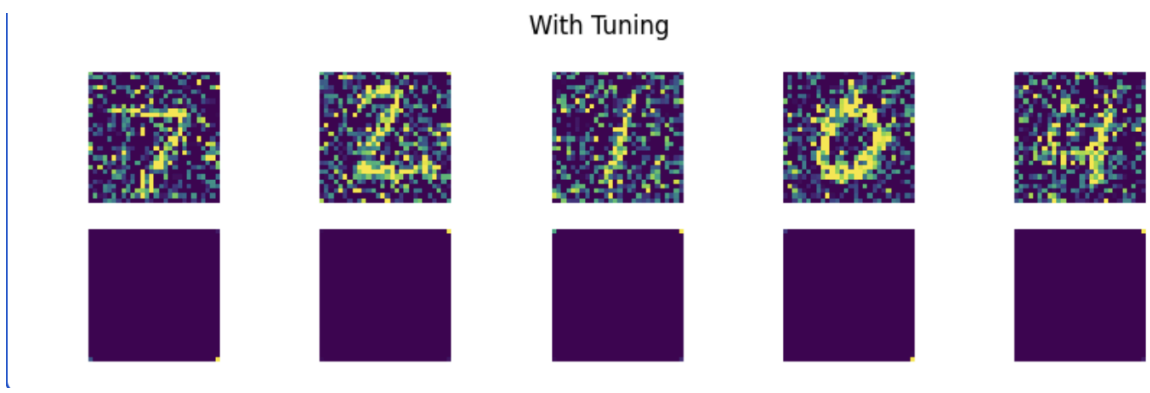
```
    # Denoised
    plt.subplot(3,5,i+6)
    plt.imshow(decoded_imgs1[i].reshape(28,28))
    plt.axis('off')

plt.suptitle("Without Tuning")
plt.show()
```



With tuning:-

```
# Tuned Autoencoder

model2 = models.Sequential([
    layers.Conv2D(64, (3,3), activation='relu', padding='same',
input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2), padding='same'),
    layers.Conv2D(32, (3,3), activation='relu', padding='same'),
    layers.UpSampling2D((2,2)),
    layers.Conv2D(1, (3,3), activation='sigmoid', padding='same')
])

model2.compile(optimizer='adam', loss='mse')

# Train with tuned parameters
history2 = model2.fit(x_train_noisy, x_train, epochs=10, batch_size=64,
                      validation_data=(x_test_noisy, x_test))
```

```
# Predict
decoded_imgs2 = model2.predict(x_test_noisy)

# Show images
plt.figure(figsize=(10,4))
for i in range(5):
    # Noisy
    plt.subplot(3,5,i+1)
    plt.imshow(x_test_noisy[i].reshape(28,28))
    plt.axis('off')

    # Denoised
    plt.subplot(3,5,i+6)
    plt.imshow(decoded_imgs2[i].reshape(28,28))
    plt.axis('off')

plt.suptitle("With Tuning")
plt.show()
```



Interpretation:-

The output shows noisy and reconstructed images. The autoencoder learns to remove noise and reconstruct clean images. The tuned model produces clearer and more accurate images compared to the untuned model.

CONCLUSION:-

Autoencoders are effective for image denoising tasks. They learn compressed representations and reconstruct clean images from noisy inputs. Hyperparameter tuning improves reconstruction quality and reduces noise more effectively. Overall, autoencoders provide strong performance in image processing applications.